

目 录

一、引言

- 1.1 编写目的
- 1.2 软件概述
- 1.3 运行环境
- 1.4 术语与缩略语

二、软件总体设计

- 2.1 软件需求概括
- 2.2 总体架构设计
- 2.3 模块划分与关系
- 2.4 场景系统设计

三、核心模块详细设计

- 3.1 游戏入口模块
- 3.2 场景管理模块
- 3.3 棋盘与消除模块
- 3.4 关卡闯关模式模块
- 3.5 经典无尽模式模块
- 3.6 道具、皮肤与排行榜模块
- 3.7 云同步与数据上报模块

四、数据结构设计

五、数据接口设计

六、出错处理设计

七、性能优化设计

一、引言

1.1 编写目的

编写本设计说明书是软件开发过程的重要组成部分。本文档旨在详细描述深圳幸运呱科技有限公司彩珠五连小游戏软件（以下简称“本软件”）的软件架构设计、核心模块设计、数据结构设计、接口设计及出错处理设计，为软件著作权登记提供技术性文档依据。

本文档面向软件著作权审查人员，全面展示本软件的技术架构、设计思路和实现方案，证明本软件为独立开发的原创作品。

1.2 软件概述

本软件是一款基于微信小游戏平台的休闲益智类连线消除小游戏。玩家在 9x9 棋盘上每回合放置三颗彩色珠子，通过形成五连或以上同色连线进行消除并得分。软件包含关卡闯关、经典无尽、皮肤主题

、道具辅助、好友排行榜、云端存档同步和玩法数据分析等功能。

本软件的主要功能包括:

- * 关卡闯关模式: 按关卡配置完成目标分数或消除任务, 支持计时、星级评价和关卡教学
- * 经典无尽模式: 在空棋盘上持续放置珠子, 通过五连消除累积得分并挑战历史最高分
- * 棋盘消除系统: 9x9 棋盘、七色珠子、五连判定、连锁消除和预览下一批珠子
- * 道具系统: 提供刷新、撤销、炸弹等辅助道具, 支持关卡内消耗和经典模式救场
- * 皮肤系统: 支持切换棋盘背景与珠子主题, 增强视觉个性化
- * 排行榜系统: 展示经典模式好友榜, 结合开放数据域渲染好友成绩
- * 云同步系统: 通过 caizhu-api 保存关卡进度、经典最高分、皮肤和道具库存
- * 数据分析系统: 上报模式进入、关卡结果、经典结束、道具使用等行为事件
- * 社交分享功能: 支持微信分享卡片和群互动能力

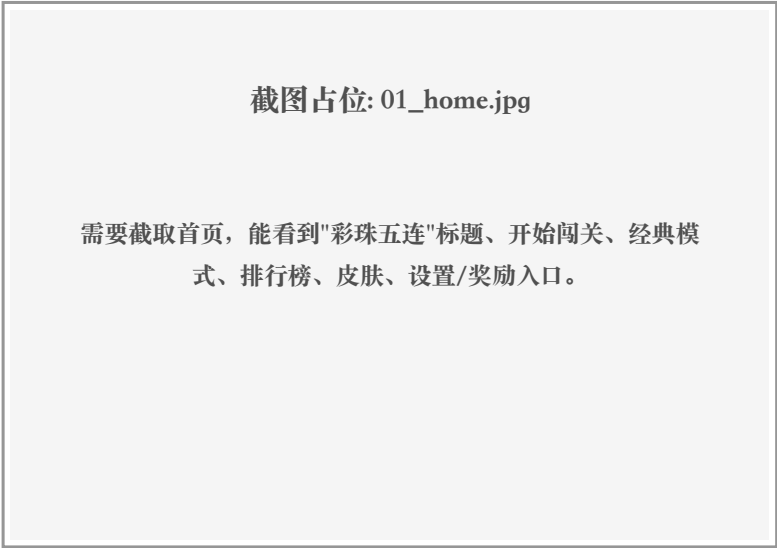


图1 首页主界面 - 玩法入口与底部功能按钮

1.3 运行环境

项目	要求
运行平台	微信小游戏
操作系统	iOS 10.0及以上 / Android 5.0及以上
微信版本	微信客户端 6.7.2 及以上
屏幕方向	竖屏(Portrait)
开发语言	TypeScript / JavaScript (ES6+)
渲染技术	PixiJS + WebGL/Canvas
构建工具	Vite
云服务	CloudBase 云函数 caizhu-api
网络要求	核心玩法可本地运行, 云同步、排行榜、数据上报需要网络连接

1.4 术语与缩略语

术语/缩略语	含义说明
五连	棋盘上连续五颗或以上同色珠子形成的可消除连线
预览区	显示下一批待放置珠子的区域，帮助玩家提前规划
经典模式	无固定关卡终点，以尽可能高的得分为目标的持续挑战玩法
关卡模式	按 LevelConfig 配置目标分数、时间和特殊规则的闯关玩法
PixiJS	用于小游戏渲染的 2D WebGL 渲染引擎
Scene	游戏场景，负责独立界面生命周期、渲染和输入处理
DPR	Device Pixel Ratio，设备像素比
SDK	Software Development Kit，软件开发工具包
API	Application Programming Interface，应用程序接口

二、软件总体设计

2.1 软件需求概括

本软件采用模块化的软件设计方法，以微信小程序框架为基础平台，使用 TypeScript 编写核心逻辑，通过 Vite 构建为小游戏可运行的 JavaScript bundle。软件采用单页面应用架构，使用 SceneManager 切换加载页、首页、关卡选择、关卡、经典、排行榜和皮肤等功能界面。

- * 高性能实时渲染: 棋盘包含大量珠子精灵、消除动画和 UI 叠加，需要稳定帧率
- * 准确触摸输入: 支持点击格子放置珠子、点击道具、弹窗按钮和排行榜区域交互
- * 可靠进度存储: 关卡进度、经典最高分、皮肤选择和道具库存需要本地持久化并可云端同步
- * 可扩展内容配置: 关卡、布局、道具、皮肤和音频路径均采用配置化管理
- * 稳定网络降级: 广告、排行、云同步和数据分析失败时不影响核心玩法继续运行

2.2 总体架构设计

本软件采用"启动入口 + 引擎核心 + 场景系统 + 管理器模块 + 配置数据 + 服务层"的分层架构。入口层负责初始化 Canvas、PixiJS、音频、分享、云同步和场景注册；Game 类封装渲染舞台、逻辑尺寸、Ticker 和开放数据域合成；业务场景负责具体玩法和界面逻辑。

模块层	模块名称	功能简述
入口层	minigame/game.js / src/main.ts	创建小游戏运行环境，初始化 Game 并注册场景
核心层	core/Game.ts	封装 PixiJS 应用、逻辑尺寸、双 Canvas 合成和 Ticker
核心层	core/SceneManager.ts	统一管理场景注册、切换和生命周期
场景层	scenes/HomeScene.ts	首页入口、玩法按钮和功能导航
场景层	scenes/LevelSelectScene.ts	关卡列表与进度展示
场景层	scenes/LevelScene.ts	关卡玩法、计时、道具和通关/失败流程
场景层	scenes/ClassicScene.ts	经典无尽玩法与结算

场景层	scenes/RankScene.ts	排行榜与开放数据域 overlay
场景层	scenes/SkinScene.ts	皮肤选择与预览
管理器	managers/BoardManager.ts	棋盘状态、放置、消除和连锁逻辑
管理器	managers/*.ts	关卡、道具、皮肤、排行、云同步和用户资料
系统层	systems/LineDetector.ts	五连及以上同色连线检测
云函数	cloudfunctions/caizhu-api	鉴权、存档、排行榜和用户资料接口

2.3 模块划分与关系

```
minigame/game.js
-> minigame/game-bundle.js
-> src/main.ts
-> core/Game.ts
-> core/SceneManager.ts
-> managers/CloudSyncManager.ts
-> scenes/LoadingScene.ts / HomeScene.ts
-> scenes/LevelSelectScene.ts / LevelScene.ts
-> scenes/ClassicScene.ts / RankScene.ts / SkinScene.ts
-> managers/BoardManager.ts / LevelManager.ts / PropManager.ts
-> gameobjects/BoardView.ts / BallSprite.ts
-> config/*.ts / ui/*.ts / analytics/index.ts
```

各场景通过 SceneManager 解耦，进入场景时执行 onEnter，离开场景时执行 onExit。BoardManager 作为棋盘逻辑核心被 LevelScene 和 ClassicScene 复用；PersistService、BackendService 和 CloudSyncManager 提供统一的数据读写与同步能力。

2.4 场景系统设计

场景	功能描述	关键模块
loading	资源加载与健康游戏忠告	LoadingScene
home	首页与玩法入口	HomeScene
levelSelect	关卡选择	LevelSelectScene
level	关卡闯关玩法	LevelScene
classic	经典无尽玩法	ClassicScene
rank	排行榜	RankScene
skin	皮肤选择	SkinScene

三、核心模块详细设计

3.1 游戏入口模块

游戏入口由 minigame/game.js 与 src/main.ts 协同完成。game.js 作为微信小程序启动文件，创建主 Canvas 并加载 Vite 构建后的 game-bundle.js；main.ts 初始化 Game、配置分享、初始化 analytics、预热

CloudSyncManager、加载分包资源、注册全部场景，并切换到 LoadingScene 或 HomeScene。

```
Game.init(canvas)
configureWechatShare()
initAnalytics()
CloudSyncManager.prewarm()
SceneManager.register(loadingScene)
SceneManager.register(homeScene)
SceneManager.switchTo("loading")
```

3.2 场景管理模块

SceneManager 维护场景注册表和当前场景引用。切换场景时先调用旧场景 onExit 清理定时器、原生按钮和临时节点，再调用新场景 onEnter 挂载界面，确保同一时间只有一个业务场景响应输入。Game.ticker 统一驱动帧更新，EventBus 用于棋盘事件在 UI 层和业务层之间解耦。

3.3 棋盘与消除模块

BoardManager 负责维护 9x9 棋盘数组、当前待放置珠子队列、放置合法性校验、LineDetector 五连检测、消除得分计算、空位填充和连锁判定。BoardView 与 BallSprite 负责将棋盘状态渲染为 Pixi 精灵，并播放放置、消除和下落动画。

子模块	功能说明
BoardManager	维护棋盘数组、放置三颗珠子、触发消除和更新得分
LineDetector	扫描横、竖、斜向连续同色珠子，返回待消除坐标集合
PathFinder	为放置预览和教学引导提供路径辅助计算
BoardView	根据棋盘状态创建/更新 BallSprite 并处理点击映射
PreviewPanel	展示下一批待放置珠子的颜色与顺序

```
placeBalls(points[])
-> 写入棋盘并扣除预览队列
-> LineDetector.findLines(board)
-> 若有连线则消除、加分、下落填充
-> 递归检测连锁消除
-> 更新 HUD / ScorePanel 并广播事件
```

3.4 关卡闯关模式模块

LevelScene 是关卡玩法核心模块，负责读取 LevelConfig 和 LevelLayouts、初始化目标分数/时间、挂载 LevelHud、PropBar、教学引导、特殊棋子介绍、通关与失败弹窗，并在结束时上报 analytics 事件。



图2 关卡选择界面 - 章节进度与星级

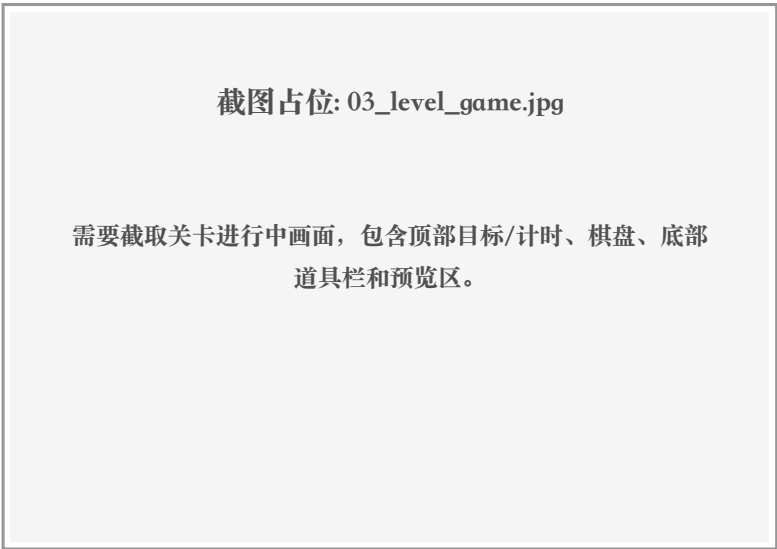


图3 关卡玩法界面 - 9x9棋盘、目标与道具栏

子模块	功能说明
LevelManager	维护当前关卡号、解锁进度和星级记录
LevelHud	展示关卡号、目标分数、倒计时和进度条
PropBar	展示刷新、撤销、炸弹等道具及库存
LevelTutorialOverlay	首关固定布局教学与手势引导
LevelCompleteOverlay	通关星级、得分和下一关入口
LevelFailOverlay	失败复活、重试和返回操作



图4 关卡通关界面 - 星级评价与下一关

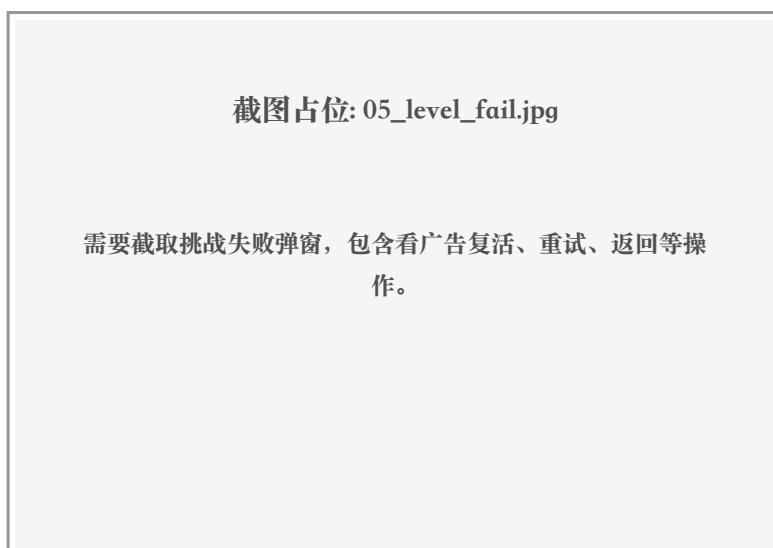


图5 关卡失败界面 - 复活与重试操作

3.5 经典无尽模式模块

ClassicScene 提供分数驱动的无尽挑战。玩家持续放置珠子并通过五连消除累积得分，棋盘无可放置位置或玩家主动结束时进入 GameOverOverlay。ClassicSaveManager 支持局内续玩，RankManager 与 LeaderboardManager 负责提交和展示经典模式好友榜成绩。

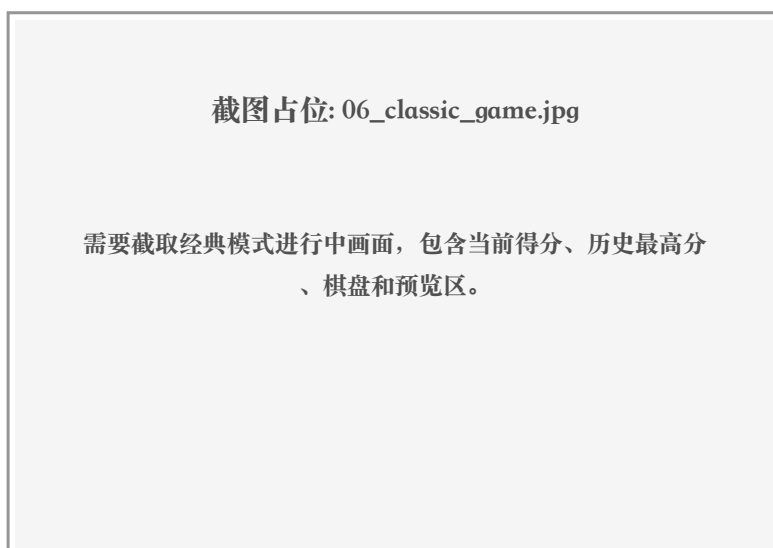


图6 经典模式界面 - 无尽得分挑战



图7 经典模式结算界面 - 本局得分与新纪录

3.6 道具、皮肤与排行榜模块

PropManager 管理道具库存和本局消耗；SkinManager 管理当前皮肤与解锁状态；RankScene 通过 Game.setOpenDataOverlay 将开放数据域 sharedCanvas 合成到主屏，展示好友排行榜。



图8 排行榜界面 - 好友榜与成绩展示

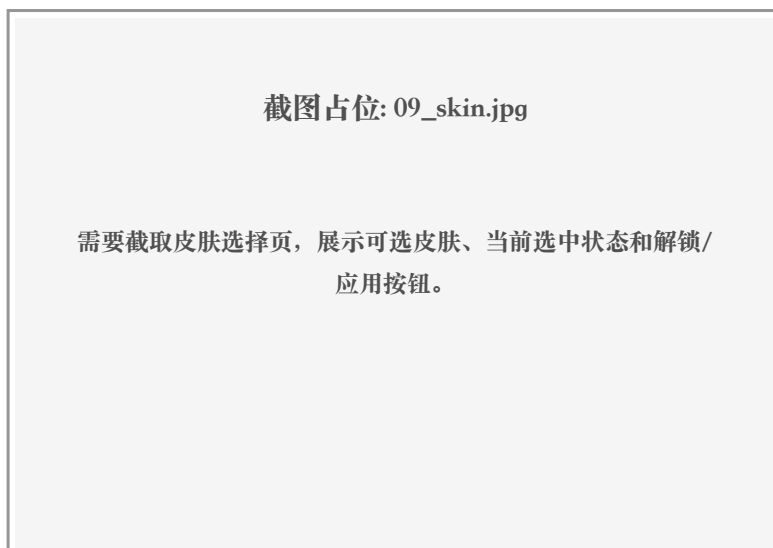


图9 皮肤界面 - 棋盘与背景主题切换

3.7 云同步与数据上报模块

CloudSyncManager 负责启动阶段权威数据拉取、运行中的防抖推送和后台 flush。BackendService 封装 caizhu-api 的鉴权、存档和排行榜接口；analytics 模块通过@gp/analytics-sdk 将用户行为事件批量上报到经分系统。SettingsOverlay 提供 BGM/SFX 开关。

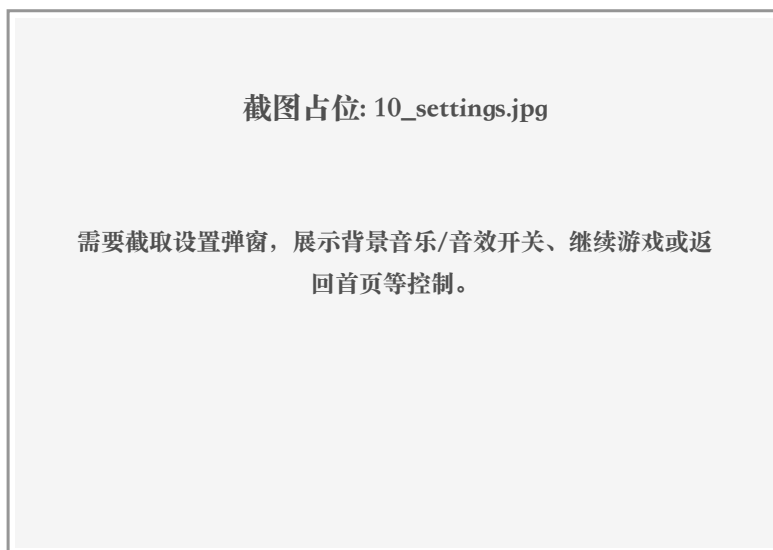


图10 设置界面 - 音频开关与返回控制

四、数据结构设计

4.1 棋盘数据结构

```
board: (number | null)[][] // 9x9, 存颜色索引或 null
previewQueue: number[] // 当前待放置的三颗珠子颜色
score: number // 当前局得分
comboChain: number // 连锁消除计数
```

4.2 关卡配置数据结构

```
interface LevelDef {
  id: number
  passScore: number
  timeLimit?: number
  starScores: [number, number, number]
  allowedProps: PropType[]
  layoutId?: string
}
```

4.3 玩家进度数据结构

```
levelProgress: { currentLevel, starsByLevel }
classicBestScore: number
skinId: string
propInventory: Record<PropType, number>
settings: { bgmOn, sfxOn }
```

4.4 云端数据结构

数据类型	关键字段	用途
用户存档	userId、snapshot、updatedAt	云端保存关卡与经典进度
排行榜记录	board、score、profile	展示经典模式好友榜
分析事件	event_name、user_id、payload	统计玩法行为和转化数据

五、数据接口设计

5.1 本地存储接口

本地存储由 PlatformService 和 PersistService 统一封装，业务模块使用同步读写接口保存进度、设置、经典最高分、皮肤选择和道具库存。存储失败时使用默认值兜底，不阻塞游戏流程。

5.2 云服务接口

接口	功能说明
/auth/login	获取用户身份令牌并绑定平台用户标识
/save/pull	拉取云端权威存档
/save/push	推送本地最新存档快照
/leaderboard/submit	提交经典模式排行榜成绩
/leaderboard/list	获取排行榜列表
analytics-ingest	接收客户端批量行为事件

5.3 微信平台接口

本软件使用微信小游戏 Canvas、分包加载、激励视频广告、分享、开放数据域、游戏圈按钮和隐私授权等能力。所有平台调用均通过 PlatformService 和工具层封装，并在不可用或失败时静默降级。

六、出错处理设计

6.1 网络异常处理

- * 云同步失败时保留本地存档，并在下次启动或后台 flush 时重试
- * 排行榜拉取失败时显示空状态或缓存数据，不影响核心玩法
- * 广告不可用时根据业务场景提示稍后重试或走不可用降级逻辑
- * analytics 上报失败时本地丢弃或下次批量重试，不阻塞游戏

6.2 数据异常处理

- * 配置缺失时使用默认值兜底，防止关卡初始化中断
- * 存档 JSON 解析失败时返回默认结构，避免坏档导致白屏
- * 无效颜色索引或纹理缺失时跳过绘制，并在控制台记录警告

- * 排行榜用户资料缺失时使用默认头像昵称展示

6.3 运行时异常处理

- * `GameGlobal.onError / onUnhandledRejection` 捕获全局异常并上报 analytics
- * 场景退出时清理 ticker 回调、原生按钮和临时容器，避免跨场景残留
- * 输入处理在动画、弹窗或教学锁定期间提前返回，避免重复触发
- * 资源加载失败时使用程序绘制兜底 UI，确保主要功能可用

七、性能优化设计

7.1 资源管理优化

- * 通过微信分包 `subpkg_assets` 拆分首页、关卡、皮肤和音频资源，降低首包体积
- * `assetPreloader` 与 `imageTexture` 统一纹理加载与缓存，避免重复创建 `Texture`
- * `orbLoader` 与 `iconLoader` 按需预加载珠子与道具图标
- * 场景退出时销毁动态创建的 `Pixi` 容器和文本节点，减少内存泄漏

7.2 渲染性能优化

- * 使用 `PixiJS` 承担精灵批处理、纹理管理和 `WebGL` 舞台渲染
- * `Game` 采用双 `Canvas` 合成方案，将开放数据域 `sharedCanvas` 与 `Pixi` 离屏渲染结果合成上屏
- * `BallSprite` 支持纹理模式与程序绘制模式切换，兼顾包体与视觉效果
- * `TweenManager` 统一管理补间动画，完成后及时移除回调

7.3 加载与包体优化

- * `Vite` 构建输出单一小游戏 bundle，便于微信小游戏加载
- * `LoadingScene` 展示健康游戏忠告和加载进度，避免白屏等待
- * `main.ts` 并行加载分包、珠子纹理和关键 UI 资源后再进入首页
- * `AudioManager` 按需创建 `InnerAudioContext`，避免音频上下文数量超限